

PART 4

Human & Multi-Agent Patterns

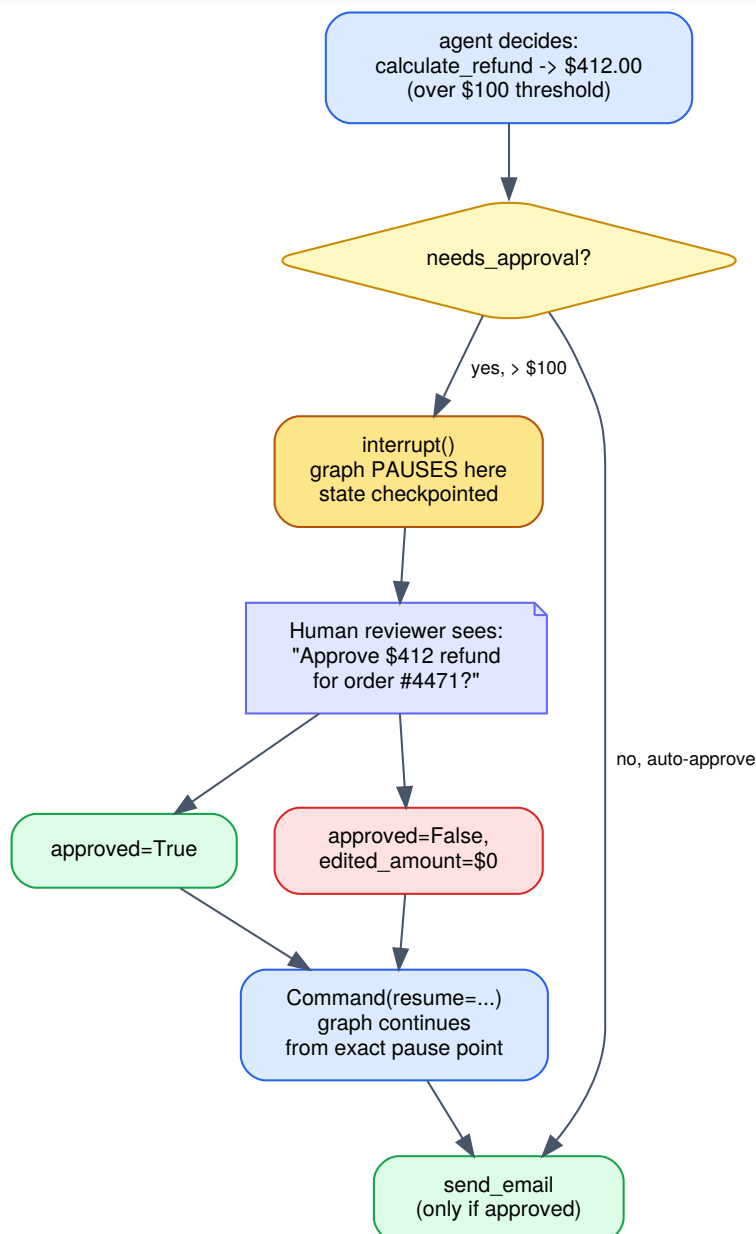
Chapters 10–12: Human-in-the-Loop, Multi-Agent Systems, Sub-graphs

Human-in-the-Loop

10.1 Conceptual Overview

Chapter 6 ended on a warning: letting a model call `send_email` automatically is fine when the content is low-stakes, but risky when it isn't. This chapter builds the actual safety mechanism — a real pause, not a "please double-check yourself" prompt instruction.

Running example — a large refund: Your support agent from Chapter 6 usually processes refunds automatically. But policy says any refund over \$100 needs a human to sign off first — a \$412 refund is not something you want an LLM approving unsupervised, even one that's usually right. When `calculate_refund` returns an amount above that threshold, the graph needs to genuinely stop, hand the decision to a person, and wait — potentially for hours — before continuing.



A human approval gate: the graph literally pauses (not polls) at `interrupt()`, and resumes exactly where it left off once a human responds.

10.1.1 `interrupt()` is a real pause, not a poll

Calling `interrupt(payload)` inside a node halts graph execution at that exact point and raises a special signal that propagates out of `.invoke()`. Crucially, this isn't the graph looping and checking "has a human responded yet?" — execution genuinely stops, and the state is checkpointed exactly as it was (this is why Chapter 7's checkpointer is a prerequisite for human-in-the-loop, not a separate feature). The process can even shut down entirely while waiting; when a human eventually responds, you

call `.invoke()` again with a `Command(resume=...)`, and the graph picks up inside the same node, at the same point, as if no time had passed.

10.1.2 Where you'd put the gate

The gate doesn't have to sit in its own dedicated node — you can call `interrupt()` directly inside the node that's about to do the risky thing (or right before it), guarded by an ordinary `if` check on the amount. This keeps the threshold logic colocated with the action it's protecting, rather than scattered across the graph.

Key idea: Human-in-the-loop is not "ask the model to be careful" — it's a structural pause enforced by your code, at a point you chose, that the model cannot talk its way past. The model doesn't decide whether to interrupt; your threshold check does.

10.2 Implementation

```
from langgraph.types import interrupt, Command

def calculate_refund_node(state: SupportState) -> dict:
    amount = calculate_refund.invoke({"order_id": state["order_id"]})
    if amount > 100:
        decision = interrupt({
            "question": f"Approve ${amount} refund for order {state['order_id']}?",
            "amount": amount,
        })
        # Execution genuinely pauses here until resumed with Command(resume=...)
        if not decision["approved"]:
            return {"messages": [("system", "Refund rejected by reviewer.")]}
        amount = decision.get("edited_amount", amount)
    return {"messages": [("system", f"Refund of ${amount} approved, proceeding.")]}

graph = builder.compile(checkpointer=checkpointer) # interrupt requires a checkpointer
config = {"configurable": {"thread_id": "cust_882"}}

# --- First call: runs until it hits interrupt(), then returns control to you ---
result = graph.invoke({"messages": [...], "order_id": "4471"}, config)
print(result["__interrupt__"])
# -> [{"question": "Approve $412 refund for order 4471?", "amount": 412}]

# --- Hours later, a human reviewer approves it in your admin UI ---
result = graph.invoke(
    Command(resume={"approved": True}),
    config,
)
# Graph resumes INSIDE calculate_refund_node, exactly after the interrupt() call,
# with `decision` bound to {"approved": True} -- not from the top of the node.
```

Note: The `thread_id` is what lets the second `.invoke()` call — potentially made from an entirely different process, hours later, after a human clicked "approve" in an admin dashboard — find its way back to the exact paused checkpoint. This is the same mechanism from Chapter 7, doing double duty.

10.3 Interview Questions

Q1. How does `interrupt()` actually pause execution — does the graph poll for a human response in a loop?

A: No polling. Calling `interrupt()` immediately halts execution at that point and surfaces the payload to the caller of `.invoke()`; the process can exit entirely. The state at that exact point is checkpointed, and execution only resumes when a separate, later `.invoke()` call is made with `Command(resume=...)` and the same `thread_id` — there's no background loop checking a queue.

Q2. Why does human-in-the-loop require a checkpointer to be configured?

A: Pausing at `interrupt()` only works if the exact state at that point can be durably saved and later reloaded — potentially after the original process has terminated. That durability is exactly what a checkpointer provides (Chapter 7); without one, there'd be no state to resume from once the process handling the first `.invoke()` call exits.

Q3. In the refund example, why is the \$100 threshold check written as plain Python rather than left to the model's judgment?

A: Trusting the model to decide "should I ask for approval here?" reintroduces exactly the risk human-in-the-loop is meant to eliminate — the model could misjudge or be persuaded to skip the check. A hard-coded threshold in your own node code guarantees the gate fires every time the condition is met, regardless of what the model reasoned about the situation.

Q4. When execution resumes after `Command(resume=...)` , does the node re-run from its first line, or continue from the interrupt point?

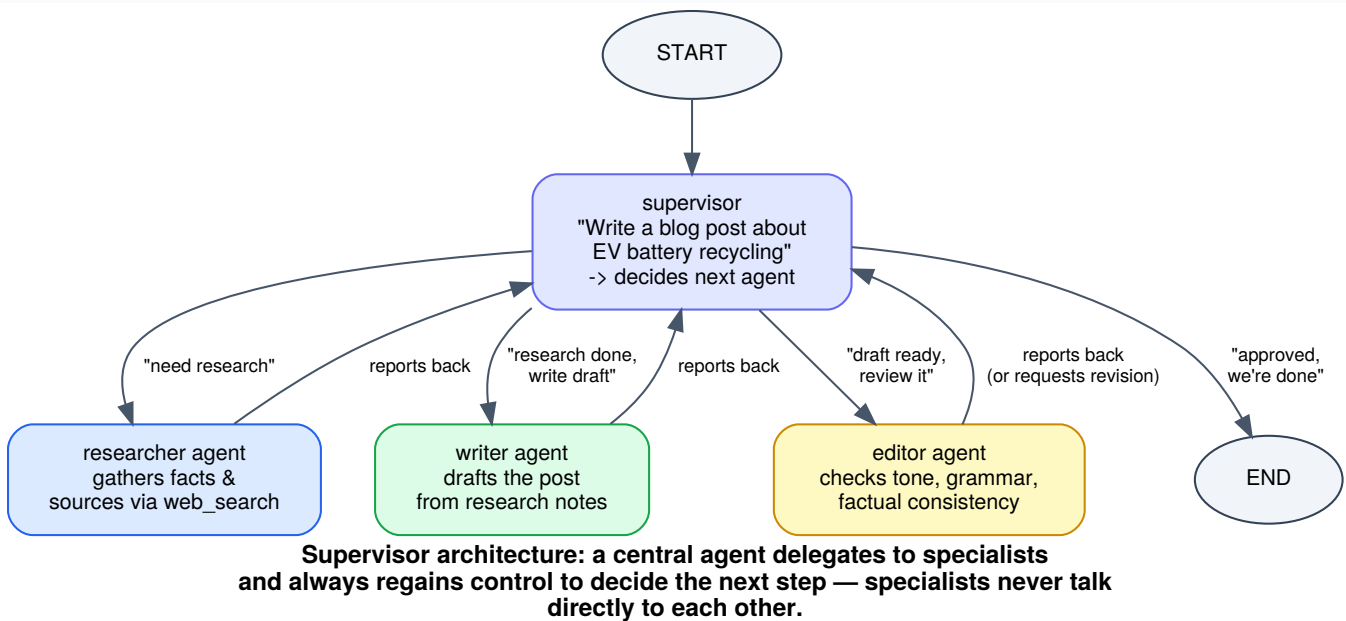
A: It continues from the interrupt point — the `interrupt()` call effectively returns the value passed to `resume` , and execution proceeds from there within the same node, rather than restarting the node from the top. This is why code before the `interrupt()` call (like the initial `calculate_refund.invoke(...)`) doesn't get re-executed.

Multi-Agent Architectures

11.1 Conceptual Overview

Every graph so far has had one "brain" — one LLM playing one role. Some tasks genuinely benefit from splitting that brain into specialists, each with a narrower job, a more focused prompt, and often a different tool set.

Running example — a content team: Someone asks for "a blog post about EV battery recycling." A single do-everything agent can technically research, write, and edit — but its prompt has to juggle three very different jobs at once, and quality suffers ("jack of all trades" agents tend to under-research and under-edit). Splitting this into a **researcher** (only gathers facts), a **writer** (only drafts, given research notes), and an **editor** (only reviews for tone/accuracy) — coordinated by a **supervisor** — mirrors how an actual content team works, and each specialist's prompt can stay short and focused.



11.1.1 The supervisor pattern

The most common and easiest-to-reason-about multi-agent architecture is **supervisor**: one central node's whole job is to look at progress so far and decide which specialist should act next. Specialists never talk to each other directly — every specialist reports back to the supervisor, and the supervisor decides the next step. This keeps control flow centralized and debuggable: at any point, "who does the supervisor think should act next?" fully explains what happens next.

11.1.2 Why not let agents talk to each other directly?

A fully peer-to-peer architecture (researcher can hand off straight to editor, editor can hand back to researcher, etc.) is more flexible in theory but far harder to debug — there's no single place that decides "what next," so tracing why the system did something requires reconstructing a conversation across many peers instead of reading one supervisor's decisions. Most production systems start with supervisor and only reach for peer-to-peer handoffs when a specific workflow genuinely needs it.

Key idea: Multi-agent isn't about the number of LLM calls — a single agent already makes many. It's about giving each LLM call a *narrower, well-defined role* with its own prompt and tools, and a supervisor whose only job is routing between them.

11.2 Implementation

```

from typing import TypedDict, Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
from pydantic import BaseModel
  
```

```

class Route(BaseModel):
    next: Literal["researcher", "writer", "editor", "FINISH"]

class TeamState(TypedDict):
    messages: Annotated[list, add_messages]
    research_notes: str
    draft: str

supervisor_llm = ChatOpenAI(model="gpt-4o").with_structured_output(Route)

def supervisor(state: TeamState) -> dict:
    decision = supervisor_llm.invoke(
        "Given the conversation so far, decide which specialist acts next, "
        "or FINISH if the post is researched, written, and edited.\n"
        f"State: research_notes={ 'set' if state.get('research_notes') else 'empty'}, "
        f"draft={ 'set' if state.get('draft') else 'empty'}\n"
        f"Messages: {state['messages']}"
    )
    return {"messages": [("system", f"Supervisor routing to: {decision.next}")] }

def route_from_supervisor(state: TeamState) -> str:
    last = state["messages"][-1].content
    return last.split(": ")[-1] if "Supervisor routing to" in last else END

researcher_llm = ChatOpenAI(model="gpt-4o") # narrow prompt, e.g. bound to web_search
def researcher(state: TeamState) -> dict:
    notes = researcher_llm.invoke(
        f"Research: {state['messages'][0].content}. Return concise bullet-point facts."
    )
    return {"research_notes": notes.content, "messages": [("system", "Research complete.")]}

writer_llm = ChatOpenAI(model="gpt-4o")
def writer(state: TeamState) -> dict:
    draft = writer_llm.invoke(
        f"Write a blog post using these notes:\n{state['research_notes']}"
    )
    return {"draft": draft.content, "messages": [("system", "Draft complete.")]}

editor_llm = ChatOpenAI(model="gpt-4o")
def editor(state: TeamState) -> dict:
    review = editor_llm.invoke(f"Review this draft for tone and accuracy:\n{state['draft']}")
    return {"messages": [("system", f"Edit complete: {review.content}")] }

builder = StateGraph(TeamState)
builder.add_node("supervisor", supervisor)
builder.add_node("researcher", researcher)
builder.add_node("writer", writer)
builder.add_node("editor", editor)

builder.add_edge(START, "supervisor")
builder.add_conditional_edges("supervisor", route_from_supervisor)
for specialist in ["researcher", "writer", "editor"]:
    builder.add_edge(specialist, "supervisor") # every specialist reports back

graph = builder.compile()

```

Note: Each specialist's LLM call is deliberately narrow — the researcher's prompt never mentions tone, the editor's prompt never mentions original research. In a real system you'd also bind different tools per specialist (e.g., only the researcher gets `web_search`), which is another reason to split roles: tool access itself becomes more precise.

11.3 Interview Questions

Q1. In the supervisor pattern, why do specialists report back to the supervisor instead of handing off directly to the next specialist?

A: Centralizing the routing decision in one place (the supervisor) means there's always a single, inspectable point that explains "what happens next and why." If specialists routed to each other directly, tracing the system's behavior would require reconstructing a distributed conversation across peers rather than reading one node's decisions — much harder to debug and reason about.

Q2. What's the actual benefit of splitting one agent into researcher/writer/editor roles, beyond "more LLM calls"?

A: Each specialist gets a narrower, more focused prompt and, typically, a narrower tool set — the researcher's prompt doesn't need to think about tone, and the editor's doesn't need to think about sourcing facts. A single do-everything prompt has to balance all these concerns at once, which in practice tends to produce weaker results at each individual sub-task than a specialist focused only on that sub-task.

Q3. How does the supervisor decide which specialist to route to next, and what stops it from looping forever between researcher and writer?

A: The supervisor is itself an LLM call given the current state (what's been done so far — research notes present? draft present?) and asked to output a structured decision (a `Route` with a constrained set of options including `FINISH`). It relies on the same kind of state-inspection and termination-condition design from Chapter 5 — the supervisor sees that a draft exists and research notes exist and can decide to move to editing or finish rather than looping, though in production you'd still want a recursion-limit safety net as in Chapter 5.

Q4. When would a peer-to-peer multi-agent architecture (specialists handing off directly to each other) actually be preferable to a supervisor?

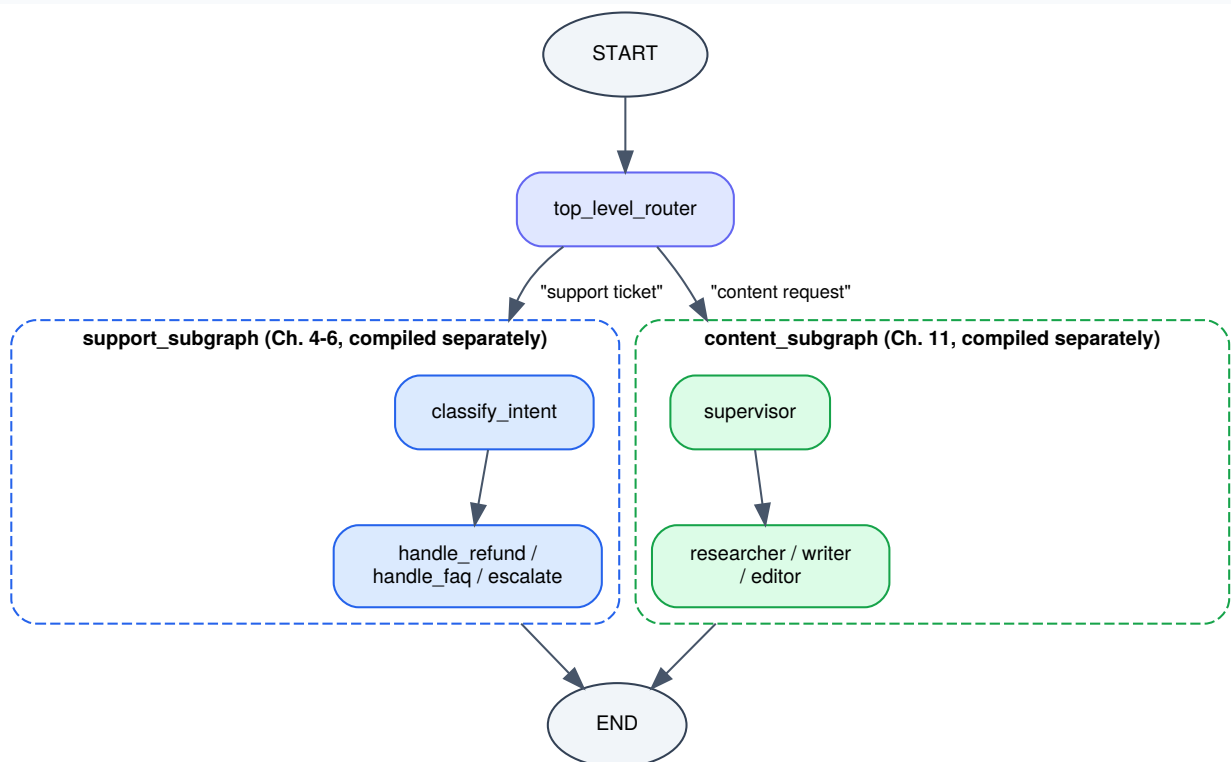
A: When the handoff logic itself is simple, fixed, and doesn't benefit from centralized judgment — e.g., a strictly linear pipeline where researcher always hands to writer always hands to editor, with no need for a "smart" router deciding between them dynamically. In that case, the supervisor's LLM call to decide routing is unnecessary overhead; a simpler pattern (or even just fixed edges) suffices, and peer-to-peer is really just a fixed pipeline in that scenario rather than a genuinely dynamic multi-agent decision.

Sub-graphs & Modular Design

12.1 Conceptual Overview

You now have two fully-built systems: the support triage agent (Chapters 4–6, 10) and the content team (Chapter 11). A real product usually needs *both*, plus a way to route between them — without smashing every node from both systems into one giant, unmanageable graph.

Running example — one company, two very different requests: Your platform receives both "my order hasn't arrived" (support) and "write me a blog post about our new product" (content) through the same front door. These are structurally unrelated workflows built by different teams, tested independently, with completely different state shapes. You want to reuse each one as-is, not rewrite them into a single monolithic graph.



Each subgraph is a fully independent compiled graph, reused as a single node in a larger parent graph — its internal nodes stay invisible to the parent.

12.1.1 A compiled graph is itself a valid node

This is the core trick: once you call `.compile()` on a `StateGraph`, the result is a runnable object — and a runnable object can be added as a node in a *different* graph via `parent_builder.add_node("support", support_graph)`. From the parent graph's point of view, the entire support triage system (however many internal nodes it has) is just one node that takes an input and produces an output. This is real encapsulation: the parent graph never sees `classify_intent` or `handle_refund` directly.

12.1.2 State shape mismatches

The support subgraph's state (`SupportState`, with `intent`, `order_id`) and the content subgraph's state (`TeamState`, with `research_notes`, `draft`) don't share a schema — and they don't need to. As long as the parent graph's state has the fields each subgraph needs (or you write a small adapter function to transform state shape at the boundary), each subgraph can keep its own internal schema exactly as it was built and tested independently.

Key idea: Subgraphs give you the same benefit as functions in ordinary software: build and test a self-contained unit once, then reuse it as a black box wherever it's needed, without its internal complexity leaking into the caller.

12.2 Implementation


```

from typing import TypedDict, Annotated, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from pydantic import BaseModel

# support_graph and content_graph are the ALREADY-COMPILED graphs
# from Chapters 6/10 and Chapter 11, respectively -- unchanged.

class TopLevelState(TypedDict):
    messages: Annotated[list, add_messages]
    request_type: str

class RequestType(BaseModel):
    kind: Literal["support", "content"]

router_llm = ChatOpenAI(model="gpt-4o-mini").with_structured_output(RequestType)

def classify_request(state: TopLevelState) -> dict:
    result = router_llm.invoke(f"Classify this request: {state['messages'][-1].content}")
    return {"request_type": result.kind}

def route_top_level(state: TopLevelState) -> str:
    return "support" if state["request_type"] == "support" else "content"

parent_builder = StateGraph(TopLevelState)
parent_builder.add_node("classify_request", classify_request)

# Compiled subgraphs plugged in directly as nodes:
parent_builder.add_node("support", support_graph)
parent_builder.add_node("content", content_graph)

parent_builder.add_edge(START, "classify_request")
parent_builder.add_conditional_edges("classify_request", route_top_level)
parent_builder.add_edge("support", END)
parent_builder.add_edge("content", END)

platform_graph = parent_builder.compile(checkpointer=checkpointer)

# A support request flows through classify_request, then into the ENTIRE
# support_graph (classify_intent -> handle_refund/faq/escalate, with its
# own interrupt() gate from Ch. 10) as if it were one atomic node.
platform_graph.invoke(
    {"messages": [("user", "My order #4471 never arrived.")]},
    {"configurable": {"thread_id": "cust_882"}},
)

```

Note: For this to work smoothly, `TopLevelState`'s `messages` field needs to be compatible with what each subgraph expects on its own `messages` field — LangGraph passes matching keys through automatically. When schemas genuinely diverge, wrap the subgraph in a small node function that translates parent state into the subgraph's expected input shape and translates its output back.

12.3 Interview Questions

Q1. What makes it possible to use an entire compiled StateGraph as a single node inside another graph?

A: Calling `.compile()` produces a runnable object with the same invocation interface any node function has (it accepts state and returns updates). Since `add_node` only requires something callable with that interface, a compiled graph qualifies just as well as a plain Python function — the parent graph treats it as one opaque unit of work.

Q2. In the platform example, does the parent graph (`platform_graph`) need to know about `classify_intent` or `handle_refund` from inside the support subgraph?

A: No — those nodes are entirely internal to `support_graph` and invisible to the parent. The parent only interacts with the subgraph as a single node named `"support"`, sending it input state and receiving output state, exactly as it would with any other node.

Q3. Why don't `SupportState` and `TeamState` need to share the same schema to both be used as subgraphs in the same parent?

A: Each subgraph manages its own internal state independently — the parent graph only needs to supply whatever keys that subgraph's entry node actually reads (commonly just `messages`) and will receive back whatever keys it writes. The

subgraphs' other internal fields (`intent` , `order_id` vs. `research_notes` , `draft`) never need to reconcile with each other because neither subgraph is ever aware the other exists.

Q4. What's the main practical benefit of building the support and content systems as separate subgraphs rather than one large combined graph with all nodes from both?

A: Each subgraph can be built, tested, versioned, and reasoned about independently — a change to the content team's editor node can't accidentally break the support agent's refund logic, and each team can own their subgraph without needing to understand the other's internals. It also means each subgraph's own control-flow features (like the support graph's `interrupt()` gate from Chapter 10) keep working unmodified when reused inside a larger system.